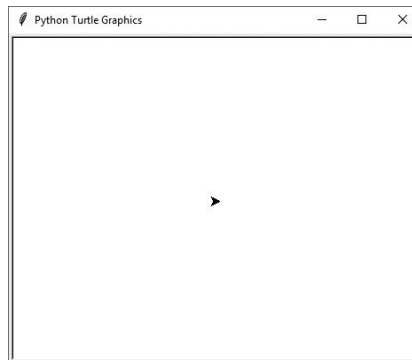


Introduction to Turtle Graphics

- **Turtle graphics is an interesting and easy way to learn basic programming concepts.**
- **The Python turtle graphics system simulates a “turtle” that obeys commands to draw simple graphics.**

Introduction to Turtle Graphics (1 of 2)

- Python's turtle graphics system displays a small cursor known as a *turtle*.



- You can use Python statements to move the turtle around the screen, drawing lines and shapes.

Introduction to Turtle Graphics (2 of 2)

- To use the turtle graphics system, you must import the turtle module with this statement:

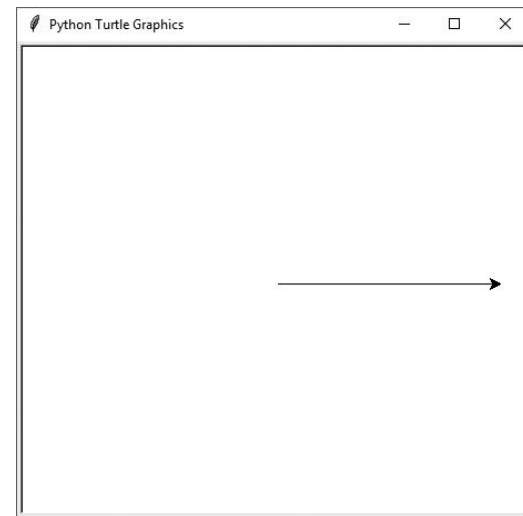
```
import turtle
```

This loads the turtle module into memory

Moving the Turtle Forward

- Use the `turtle.forward(n)` statement to move the turtle forward *n* pixels.

```
>>> import turtle
>>> turtle.forward(100)
>>>
```

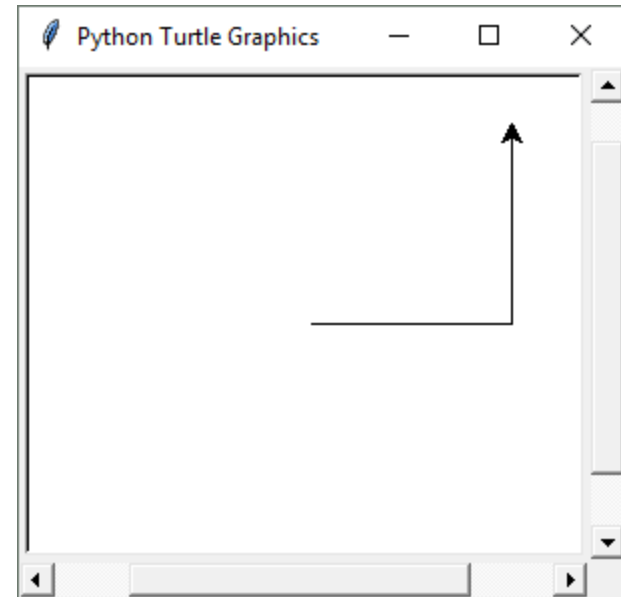


Turning the Turtle (1 of 3)

- The turtle's initial heading is 0 degrees (east)
- Use the `turtle.right(angle)` statement to turn the turtle right by *angle* degrees.
- Use the `turtle.left(angle)` statement to turn the turtle left by *angle* degrees.

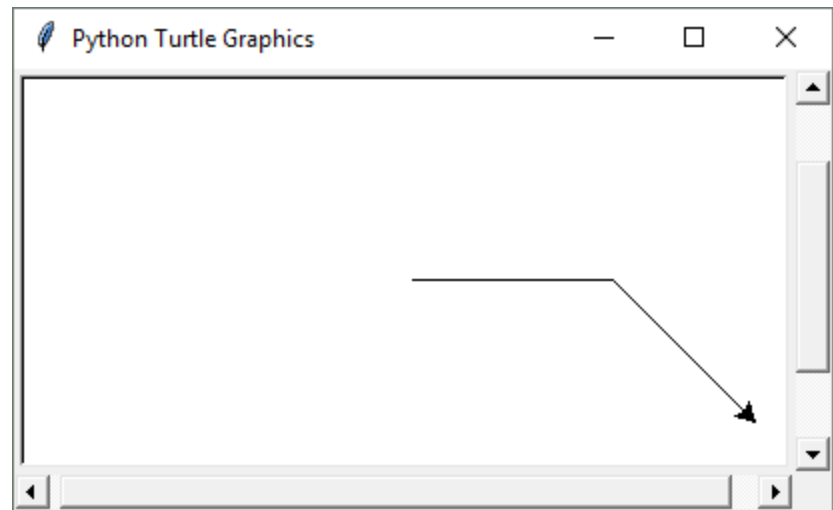
Turning the Turtle (2 of 3)

```
>>> import turtle
>>> turtle.forward(100)
>>> turtle.left(90)
>>> turtle.forward(100)
>>>
```



Turning the Turtle (3 of 3)

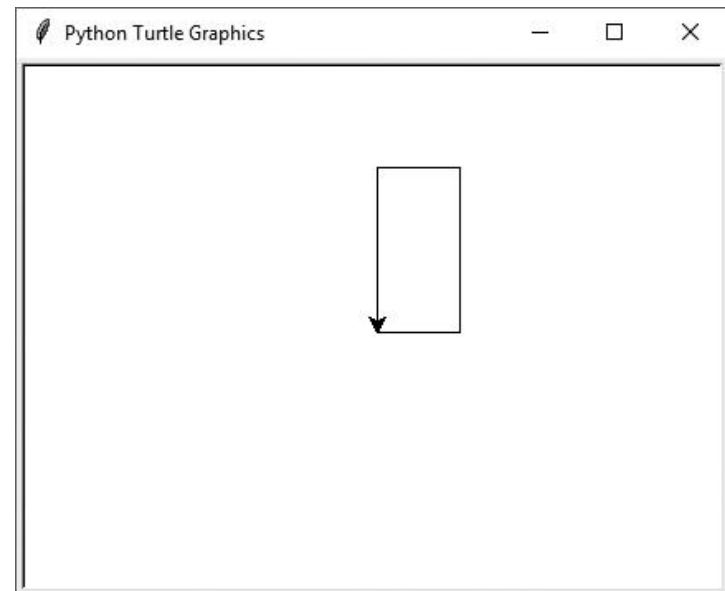
```
>>> import turtle
>>> turtle.forward(100)
>>> turtle.right(45)
>>> turtle.forward(100)
>>>
```



Setting the Turtle's Heading

- Use the `turtle.setheading(angle)` statement to set the turtle's heading to a specific angle.

```
>>> import turtle
>>> turtle.forward(50)
>>> turtle.setheading(90)
>>> turtle.forward(100)
>>> turtle.setheading(180)
>>> turtle.forward(50)
>>> turtle.setheading(270)
>>> turtle.forward(100)
>>>
```

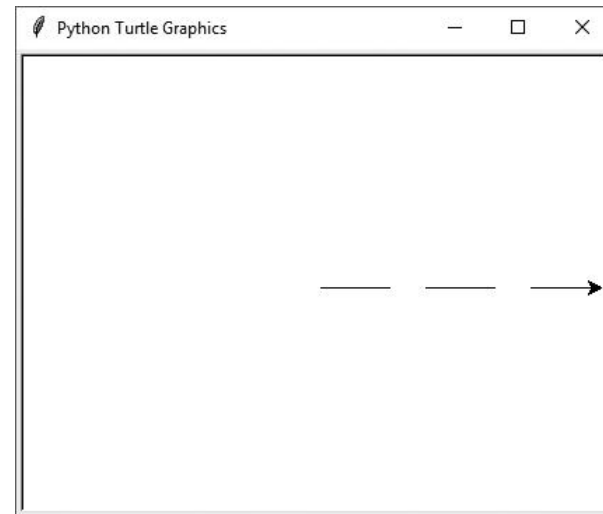


Setting the Pen Up or Down (1 of 2)

- When the turtle's pen is down, the turtle draws a line as it moves. By default, the pen is down.
- When the turtle's pen is up, the turtle does not draw as it moves.
- Use the `turtle.penup()` statement to raise the pen.
- Use the `turtle.pendown()` statement to lower the pen.

Setting the Pen Up or Down (2 of 2)

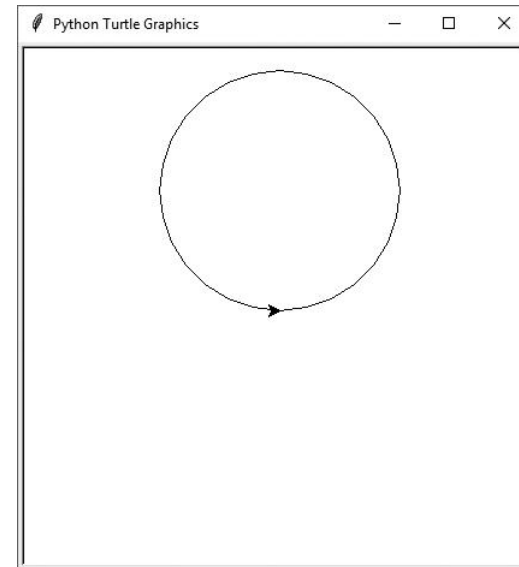
```
>>> import turtle
>>> turtle.forward(50)
>>> turtle.penup()
>>> turtle.forward(25)
>>> turtle.pendown()
>>> turtle.forward(50)
>>> turtle.penup()
>>> turtle.forward(25)
>>> turtle.pendown()
>>> turtle.forward(50)
>>>
```



Drawing Circles

- Use the `turtle.circle(radius)` statement to draw a circle with a specified radius.

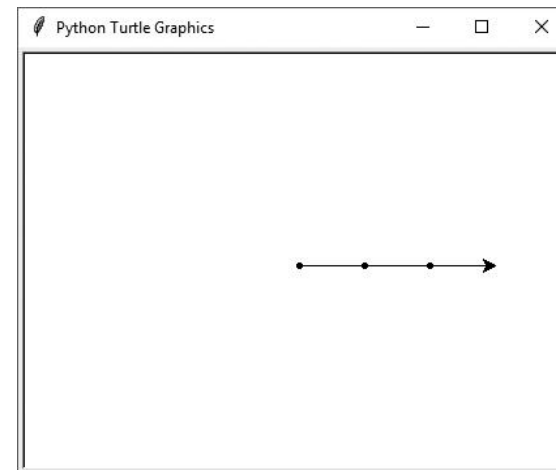
```
>>> import turtle
>>> turtle.circle(100)
>>>
```



Drawing Dots

- Use the `turtle.dot()` statement to draw a simple dot at the turtle's current location.

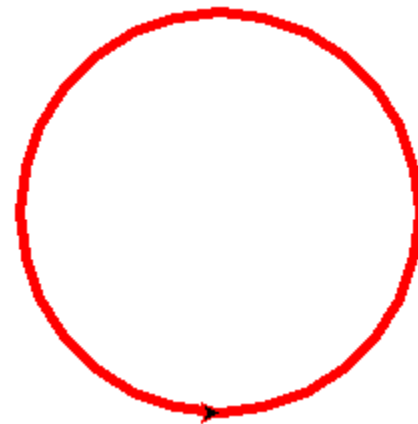
```
>>> import turtle
>>> turtle.dot()
>>> turtle.forward(50)
>>> turtle.dot()
>>> turtle.forward(50)
>>> turtle.dot()
>>> turtle.forward(50)
>>>
```



Changing the Pen Size and Drawing Color

- Use the `turtle.pensize(width)` statement to change the width of the turtle's pen, in pixels.
- Use the `turtle.pencolor(color)` statement to change the turtle's drawing color.
 - See *Appendix D* in your textbook for a complete list of colors.

```
>>> import turtle
>>> turtle.pensize(5)
>>> turtle.pencolor('red')
>>> turtle.circle(100)
>>>
```



Working with the Turtle's Window

- Use the `turtle.bgcolor(color)` statement to set the window's background color.
 - See Appendix D in your textbook for a complete list of colors.
- Use the `turtle.setup(width, height)` statement to set the size of the turtle's window, in pixels.
 - The *width* and *height* arguments are the width and height, in pixels.
 - For example, the following interactive session creates a graphics window that is 640 pixels wide and 480 pixels high:

```
>>> import turtle
>>> turtle.setup(640, 480)
>>>
```

Resetting the Turtle's Window (2 of 3)

- The `turtle.clear()` statement:
 - Erases all drawings that currently appear in the graphics window.
 - Does *not* change the turtle's position.
 - Does *not* change the drawing color.
 - Does *not* change the graphics window's background color.

Resetting the Turtle's Window (1 of 3)

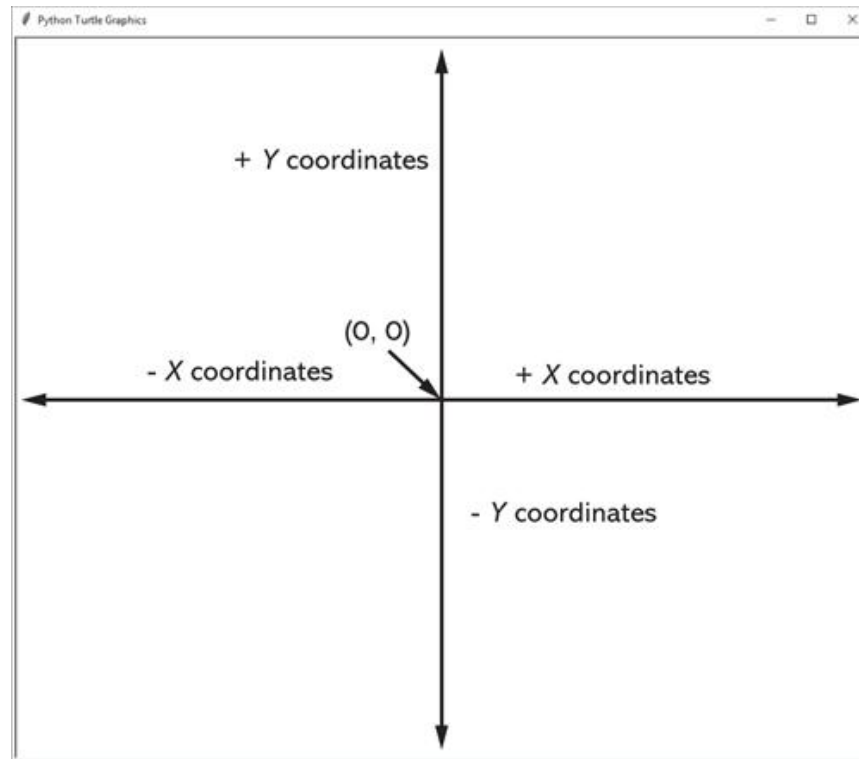
- The `turtle.reset()` statement:
 - Erases all drawings that currently appear in the graphics window.
 - Resets the drawing color to black.
 - Resets the turtle to its original position in the center of the screen.
 - Does *not* reset the graphics window's background color.

Resetting the Turtle's Window (3 of 3)

- The `turtle.clearscreen()` statement:
 - Erases all drawings that currently appear in the graphics window.
 - Resets the drawing color to black.
 - Resets the turtle to its original position in the center of the screen.
 - Resets the graphics window's background color to white.

Working with Coordinates

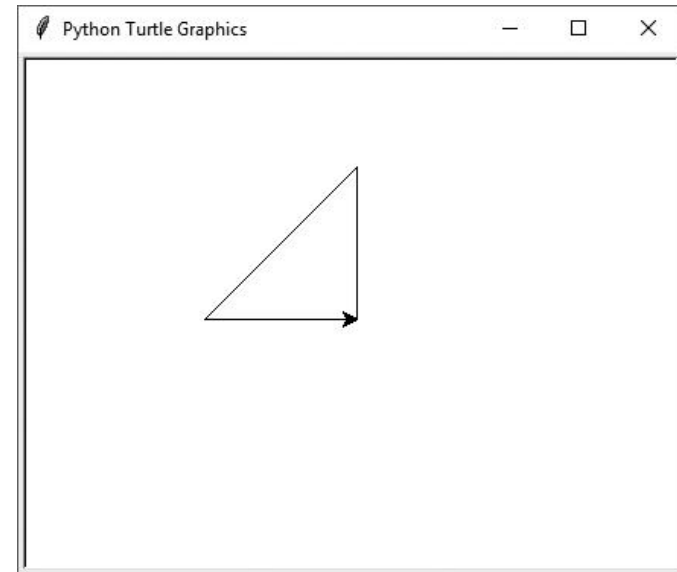
- The turtle uses Cartesian Coordinates



Moving the Turtle to a Specific Location

- Use the `turtle.goto(x, y)` statement to move the turtle to a specific location.

```
>>> import turtle
>>> turtle.goto(0, 100)
>>> turtle.goto(-100, 0)
>>> turtle.goto(0, 0)
>>>
```



- The `turtle.pos()` statement displays the turtle's current X,Y coordinates.
- The `turtle.xcor()` statement displays the turtle's current X coordinate and the `turtle.ycor()` statement displays the turtle's current Y coordinate.

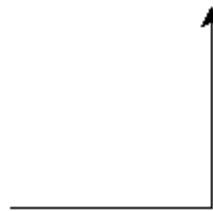
Animation Speed

- Use the `turtle.speed(speed)` command to change the speed at which the turtle moves.
 - The *speed* argument is a number in the range of 0 through 10.
 - If you specify 0, then the turtle will make all of its moves instantly (animation is disabled).

Hiding and Displaying the Turtle

- Use the `turtle.hideturtle()` command to hide the turtle.
 - This command does not change the way graphics are drawn, it simply hides the turtle icon.
- Use the `turtle.showturtle()` command to display the turtle.

```
>>> turtle.forward(100)
>>> turtle.left(90)
>>> turtle.forward(100)
```

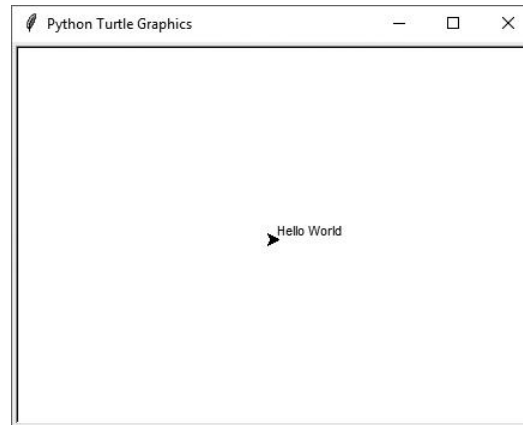


Displaying Text (1 of 2)

- Use the `turtle.write(text)` statement to display text in the turtle's graphics window.
 - The `text` argument is a string that you want to display.
 - The lower-left corner of the first character will be positioned at the turtle's *X* and *Y* coordinates.

Displaying Text (2 of 2)

```
>>> import turtle  
>>> turtle.write('Hello World')  
>>>
```

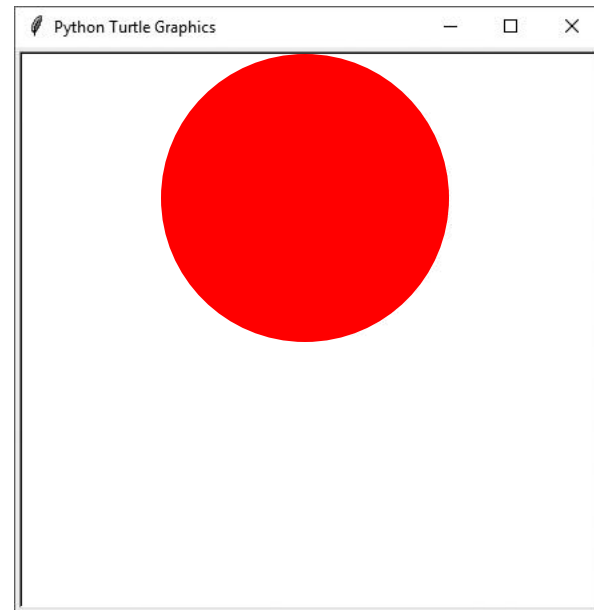


Filling Shapes (1 of 2)

- To fill a shape with a color:
 - Use the `turtle.begin_fill()` command before drawing the shape
 - Then use the `turtle.end_fill()` command after the shape is drawn.
 - When the `turtle.end_fill()` command executes, the shape will be filled with the current fill color

Filling Shapes (2 of 2)

```
>>> import turtle
>>> turtle.hideturtle()
>>> turtle.fillcolor('red')
>>> turtle.begin_fill()
>>> turtle.circle(100)
>>> turtle.end_fill()
>>>
```

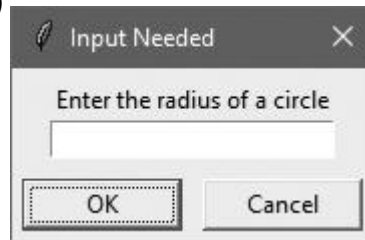


Exercise

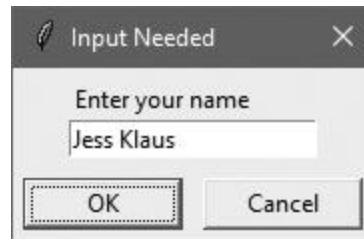
- draw a square that is filled with the color blue

Getting Input With a Dialog Box (1 of 2)

```
>>> import turtle
>>> radius = turtle.numinput('Input Needed', 'Enter the
radius of a circle')
```



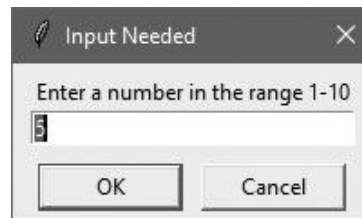
```
>>> import turtle
>>> name = turtle.textinput('Input Nedded', 'Enter your
name')
```



Getting Input With a Dialog Box (2 of 2)

- Specifying a default value, minimum value, and maximum value with `turtle.numinput`:

```
>>> import turtle
>>> num = turtle.numinput('Input Needed', 'Enter a number
in the range 1-10', default=5, minval=1, maxval=10)
```



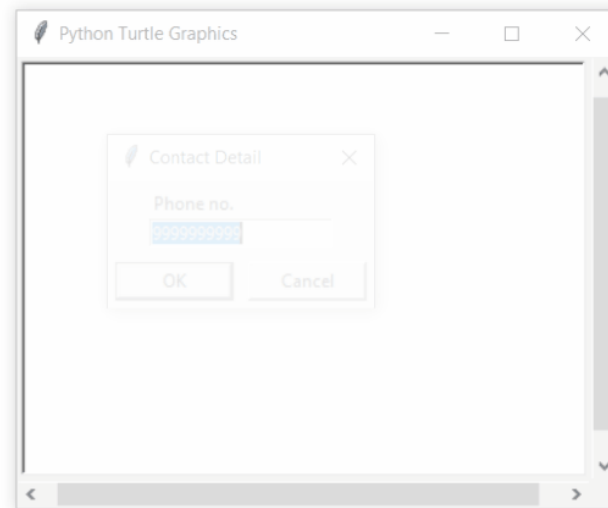
- An error message will be displayed if the input is less than `minval` or greater than `maxval`

Getting Input With a Dialog Box (2 of 2)

```
import turtle

num = int(turtle.numinput("Contact Detail", "Phone no.",
                          default=9999999999, minval=6000000000, maxval=9999999999))

print(num)
```

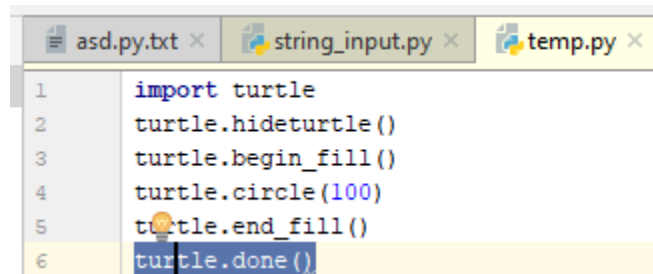


Output:

9897347846

Keeping the Graphics Window Open

- When running a turtle graphics program outside IDLE, the graphics window closes immediately when the program is done.
- To prevent this, add the `turtle.done()` statement to the very end of your turtle graphics programs.
 - This will cause the graphics window to remain open, so you can see its contents after the program finishes executing.



```
asd.py.txt x string_input.py x temp.py x
1 import turtle
2 turtle.hideturtle()
3 turtle.begin_fill()
4 turtle.circle(100)
5 turtle.end_fill()
6 turtle.done()
```

Serializing Objects

- **Serialize an object**: convert the object to a stream of bytes that
can easily be stored in a file
- **Pickling**: serializing an
object



Serializing Objects (cont'd.)

- **To pickle an object:**
 - Import the `pickle` module
 - Open a file for binary writing
 - Call the `pickle.dump` function
 - Format: `pickle.dump(object, file)`
 - Close the file
- **You can pickle multiple objects to one file prior to closing the file**

Serializing Objects (Example)

- **Demonstration of pickling a dictionary:**

```
1 >>> import pickle ↵
2 >>> phonebook = {'Chris' : '555-1111', ↵
3                 'Katie' : '555-2222', ↵
4                 'Joanne' : '555-3333'} ↵
5 >>> output_file = open('phonebook.dat', 'wb') ↵
6 >>> pickle.dump(phonebook, output_file) ↵
7 >>> output_file.close() ↵
8 >>>
```

*wb - binary
writing*

Serializing Objects (cont'd.)

- **Unpickling**: retrieving pickled object
- **To unpickle an object:**
 - Import the `pickle` module
 - Open a file for binary reading
 - Call the `pickle.load` function
 - Format: `pickle.load(file)`
 - Close the file
- **You can unpickle multiple objects from the file**

Serializing Objects (Example)

- How to unpickle a dictionary:

```
1 >>> import pickle ↵
```

```
2 >>> input_file = open('phonebook.dat', 'rb') ↵
```

```
3 >>> pb = pickle.load(input_file) ↵
```

```
4 >>> pb ↵
```

```
5 {'Chris': '555-1111', 'Joanne': '555-3333', 'Katie': '555-2222'}
```

```
6 >>> input_file.close() ↵
```

```
7 >>>
```

rb - binary
reading



Serializing Objects (Example)

- How to pickle and unpickle a set:

```
>>> import pickle
>>> dep2={'Ran', 'Nir', 'Din', 'Idan', 'Orit'}
>>> output_file = open('dep2.dat', 'wb')
>>> pickle.dump(dep2, output_file)
>>> output_file.close()
>>> input_file = open('dep2.dat', 'rb')
>>> pb = pickle.load(input_file)
>>> pb
{'Ran', 'Idan', 'Orit', 'Nir', 'Din'}
>>>
```

rb - binary
reading

